JAVA PRACTICAL PROJECT RECORD

Bank Account Management System

Program file: BankAccountManagement.java
Language: Java
Account number: 101
Account holder: Anu
Initial balance: 5000.0

Contents	Aim, requirements, concepts, algorithm, program flow, full source code, code explanation, compile/run commands, five test cases with corrected console outputs, summary, limitations, and conclusion.

Output note: The console transcripts explicitly show the numbers entered by the user beside the prompts. Each case is a separate run and starts with a fresh balance of 5000.0.

1. Aim

To create a simple bank account program using a class, constructor, object, and methods to display account details, deposit money, and withdraw money with an insufficient-balance check.

2. Requirements

• Java Development Kit (JDK), including javac and java
• Text editor or Java IDE
• Terminal or command prompt

3. Concepts Used

Class and object; instance variables; constructor; this keyword; Scanner input; methods; arithmetic updates; if-else decision making.

4. Algorithm

1. Create a Scanner to read keyboard input.

2. Create a BankAccount object with account number 101, holder name Anu, and initial balance 5000.

3. Display the account details.

4. Read the deposit amount and add it to the balance.

5. Print the deposit success message and updated balance.

6. Read the withdrawal amount.

7. If the withdrawal amount is less than or equal to balance, subtract it; otherwise print Insufficient Balance.

8. Close Scanner and end.

5. Program Flow

Start
Create Scanner and BankAccount object
Display account details
Read deposit amount → add to balance
Read withdrawal amount
Check whether withdrawal ≤ balance
Yes: subtract amount and display balance
No: display Insufficient Balance
Close Scanner → End

6. Complete Source Code

```
import java.util.Scanner;
class BankAccount {
    int accountNumber;
    String accountHolder;
    double balance;
    BankAccount(int accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }
    void displayAccount() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: " + balance);
    }
    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully");
        System.out.println("New Balance: " + balance);
    }
    void withdraw(double amount) {
        if (amount &lt;= balance) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully");
            System.out.println("New Balance: " + balance);
        } else {
            System.out.println("Insufficient Balance");
        }
    }
}
public class BankAccountManagement {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        BankAccount account =
            new BankAccount(101, "Anu", 5000);
        account.displayAccount();
        System.out.print("Enter deposit amount: ");
        double depositAmount = sc.nextDouble();
        account.deposit(depositAmount);
        System.out.print("Enter withdrawal amount: ");
        double withdrawAmount = sc.nextDouble();
        account.withdraw(withdrawAmount);
        sc.close();
    }
}
```

7. Code Explanation

Code element	Explanation
BankAccount class	Defines the account's data and operations.
Instance variables	accountNumber, accountHolder, and balance store data for the account object.
Constructor	Initializes the account fields when a BankAccount object is created.
this keyword	Refers to the object's field when a constructor parameter has the same name.
displayAccount()	Prints the account number, holder name, and current balance.
deposit(amount)	Adds the deposit amount to balance and prints the updated balance.
withdraw(amount)	Withdraws if amount <= balance; otherwise prints Insufficient Balance.
Scanner and nextDouble()	Read the deposit and withdrawal values entered by the user.
main()	Creates the account, calls methods in sequence, and closes Scanner.

8. Compilation and Execution

```
javac BankAccountManagement.java
java BankAccountManagement
```

Compilation status in the available Java environment: PASS

9. Test Cases and Corrected Console Outputs

The code was compiled and each scenario was run separately. The entered values are shown after each input prompt for readability, as they appear in an interactive terminal.

Case 1 — Normal deposit and withdrawal — PASS

Deposit 1000 increases the balance to 6000; withdrawal 2000 leaves 4000. **Inputs:** deposit=1000; withdrawal=2000.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: 1000
Amount Deposited Successfully
New Balance: 6000.0
Enter withdrawal amount: 2000
Amount Withdrawn Successfully
New Balance: 4000.0
```

Case 2 — Zero deposit and withdrawal — PASS

Both amounts are zero, so the balance remains 5000. **Inputs:** deposit=0; withdrawal=0.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: 0
Amount Deposited Successfully
New Balance: 5000.0
Enter withdrawal amount: 0
Amount Withdrawn Successfully
New Balance: 5000.0
```

Case 3 — Withdraw the entire available balance — PASS

Withdrawal equal to the balance is accepted; final balance is 0. **Inputs:** deposit=0; withdrawal=5000.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: 0
Amount Deposited Successfully
New Balance: 5000.0
Enter withdrawal amount: 5000
Amount Withdrawn Successfully
New Balance: 0.0
```

Case 4 — Insufficient balance — PASS

Withdrawal exceeds the available balance and is rejected. **Inputs:** deposit=0; withdrawal=6000.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: 0
Amount Deposited Successfully
New Balance: 5000.0
Enter withdrawal amount: 6000
Insufficient Balance
```

Case 5 — Deposit then withdraw the full balance — PASS

Deposit raises balance to 6500; withdrawing 6500 leaves 0. **Inputs:** deposit=1500; withdrawal=6500.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: 1500
Amount Deposited Successfully
New Balance: 6500.0
Enter withdrawal amount: 6500
Amount Withdrawn Successfully
New Balance: 0.0
```

10. Test Summary

Case	Deposit	Withdrawal	Expected / observed result	Status
Case 1	1000	2000	Deposit 1000 increases the balance to 6000; withdrawal 2000 leaves 4000.	PASS
Case 2	0	0	Both amounts are zero, so the balance remains 5000.	PASS
Case 3	0	5000	Withdrawal equal to the balance is accepted; final balance is 0.	PASS
Case 4	0	6000	Withdrawal exceeds the available balance and is rejected.	PASS
Case 5	1500	6500	Deposit raises balance to 6500; withdrawing 6500 leaves 0.	PASS

11. Limitations and Improvements

The given code does not reject negative transaction amounts. A negative deposit reduces the balance, and a negative withdrawal may increase it. The program performs one deposit and one withdrawal per execution and does not save account data after it exits. A safer version should validate that each transaction amount is greater than zero.

12. Result and Conclusion

The BankAccountManagement program demonstrates creating an account object, displaying account details, depositing funds, and preventing withdrawals that exceed the available balance. All five listed scenarios were compiled and executed successfully in the available Java environment. The corrected output transcript includes the entered values after the input prompts.